



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**Greenpharm: A Green Cloud-Based Pharmacy Inventory Management System
For Sustainable Inventory Optimization**

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements

for the Course of

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

D S Tharuniya Priya (192511317)

Under the Supervision of

Dr. C. Rajesh Babu

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “GreenPharm: A Green Cloud-Based Pharmacy Inventory Management System for Sustainable Inventory Optimization” has been carried out by D S Tharuniya Priya (Reg. No. 192511317) , V Sanjana Sri (Reg. No. 192511084) and S K Gobika (Reg. No. 192521463) under the supervision of Dr. C. Rajesh Babu and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Computer Science and Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR	COURSE FACULTY
Dr. A. Gnana Soundari Professor CSE/Machine Learning SIMATS Engineering SIMATS, Chennai - 602105	Dr C. Rajesh Babu Professor CSE/ Big Data and Network Security SIMATS Engineering SIMATS, Chennai - 602105

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

We, D S Tharuniya Priya, S K Gobika and V Sanjana Sri of the Computer Science and Engineering Department, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “GreenPharm: A Green Cloud-Based Pharmacy Inventory Management System for Sustainable Inventory Optimization” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: THANDALAM

Date:

Signature of the Students with Names
D.S.THARUNIYA PRIYA(192511317)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
D S Tharuniya Priya 192511317	Complete system implementation, presentation & documentation	Implemented the complete GreenPharm application, including frontend, backend, database integration and system functionality	Performed/handled functional testing, validation and implementation verification	Prepared individual report/documentation sections and contributed to the project presentation	50%
V Sanjana Sri 192511084	Presentation preparation & documentation	Assisted in presenting and documenting the implemented system through the project presentation	-	Prepared individual report/documentation sections and contributed to group presentations	25%
S K Gobika 192521463	Individual documentation & report preparation	-	-	Prepared individual report/documentation sections	25%
Total					100%

ABSTRACT

Pharmacy inventory operations require accurate control of medicine quantities, batches, suppliers, purchase records and expiry dates. Manual or spreadsheet-based tracking can make low-stock and near-expiry conditions difficult to identify in time, increasing the risk of stockouts, overstocking and pharmaceutical waste. GreenPharm addresses this engineering problem through a modular cloud-based pharmacy inventory management system implemented with a React frontend, FastAPI backend and PostgreSQL database. The system centralizes medicine, supplier, stock and purchase information and provides expiry monitoring, low-stock detection, reorder suggestions, notifications, reporting and dashboard analytics. Role-based login separates pharmacist, inventory-manager and administrator identities, while server-side inventory services enforce stock-movement rules and expiry actions. Green-computing considerations are implemented through targeted aggregation queries and application-level processing rather than an unnecessarily heavy real-time analytics pipeline. The development dataset contains 24 medicine records, 6 suppliers, 3 users, 20 stock transactions, 10 purchase orders, 6 notifications and 2 expiry records. A snapshot of the bundled development dataset on 03 September 2026 contains 13 in-stock, 4 low-stock, 2 out-of-stock, 2 expiring-soon and 3 expired medicine records. The project presentation reports that all test near-expiry/expired records were correctly flagged and that dashboard values were correctly updated after test transactions. The current system is therefore best described as a validated functional prototype suitable for pilot testing. Production readiness still requires larger-scale load testing, a real-pharmacy data pilot, stronger deployment monitoring and completion of backend authorization hardening.

Keywords: *Pharmacy Inventory, Cloud Computing, FastAPI, PostgreSQL, Expiry Monitoring, Green Computing*

TABLE OF CONTENTS

ABSTRACT	5
TABLE OF CONTENTS	6
LIST OF FIGURES	9
LIST OF TABLES	10
LIST OF ABBREVIATIONS / SYMBOLS	11
CHAPTER 1	12
INTRODUCTION AND ENGINEERING PROBLEM	12
1.1 Background.....	12
1.2 Need for the Project.....	12
1.3 Problem Statement.....	13
1.4 Project Aim.....	13
1.5 Project Objectives.....	13
1.6 Scope	13
1.7 Expected Engineering Outcome	14
CHAPTER 2.....	15
LITERATURE REVIEW AND EXISTING SOLUTIONS.....	15
2.1 Review Method	15
2.2 Review of Existing Work	15
2.3 Comparative Analysis	16
2.4 Research / Engineering Gap	16
2.5 Proposed Contribution.....	16
CHAPTER 3	17
ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY.....	17
3.1 Requirements.....	17
Stakeholder / User Requirements	17
Functional & Non-Functional Requirements	17

3.2 Constraints & Applicable Standards	18
3.3 Design Specifications	19
3.4 Alternative Solutions & Evaluation.....	19
3.5 Selected Design & Detailed Design	20
3.6 Trade-off Analysis.....	21
3.7 Sustainability, Ethics, Safety, Risk & Security	22
Risk Register	23
CHAPTER 4.....	24
IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT	24
4.1 Development Approach & Resources	24
4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used	24
Software Implementation	25
Implementation Evidence from Source	26
4.3 Test Plan & Verification.....	26
4.4 Results	27
4.5 Data Analysis & Engineering Judgment	28
4.6 Comparison with Existing Solutions & Achievement of Objectives	29
4.7 Strengths & Limitations	29
4.8 Project Planning, Roles & Budget.....	30
CHAPTER 5.....	31
CONCLUSION, FUTURE WORK AND REFLECTION	31
5.1 Conclusion.....	31
5.2 Future Work.....	31
5.3 Professional Learning & Individual Reflection.....	31
New Knowledge Acquired	31
Engineering & Professional Skills Developed	32
Individual Reflection	32
REFERENCES.....	34
APPENDICES.....	35

Appendix A – Detailed Calculations	35
Appendix B – Engineering Drawings / Schematics	36
Appendix C – Source Code / Code Repository Information	36
Appendix D – Datasheets	37
Appendix E – Experimental / Raw Data	37
Appendix F – Ethics / Institutional Approval.....	39
Appendix G – Project Meeting / Progress Records.....	39
Appendix H – User/Industry Feedback	40
Appendix I – Publications / Patents / Competitions / Product Outcomes	40
Appendix J – Individual Contribution Evidence	40
CAPSTONE PROJECT OUTCOME MAPPING SHEET	42

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 1	Architecture of Greenpharm	21
Figure 2	Output Screenshot : Purchases dashboard	26
Figure 3	Output Screenshot : Suppliers dashboard	26
Figure 4	Development Dataset Status Snapshot. This is a data snapshot, not a production performance result.	28

LIST OF TABLES

Table No.	Table Name	Page No.
Table 1	Team and contribution information	12
Table 2	Literature and existing-solution comparison	17
Table 3	Stakeholder / user requirements	18
Table 4	Functional and non-functional requirements	18
Table 5	Constraints and applicable standards	19
Table 6	Design specifications	19
Table 7	Alternative-solution evaluation	20
Table 8	Trade-off analysis	22
Table 9	Risk register	22
Table 10	Test plan and verification	23
Table 11	Development dataset snapshot	26
Table 12	Project roles and responsibilities	27
Table 13	Cost table	28

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
API	Application Programming Interface	1
CRUD	Create, Read, Update, Delete	1,2
DB	Database	2
HTTPS	Hypertext Transfer Protocol Secure	3
ML	Machine Learning	5
RBAC	Role-Based Access Control	2,3
REST	Representational State Transfer	1
SPA	Single Page Application	2,3
SQL	Structured Query Language	4

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Pharmacy inventory management is an information-management problem in which medicine identity, batch information, quantity, pricing, supplier relationships, stock movements and expiry dates must remain consistent. The GreenPharm project treats the pharmacy inventory as a structured cloud application rather than a collection of paper logs or disconnected spreadsheets. The implemented solution uses a React-based web interface, a FastAPI REST backend and PostgreSQL persistence. The frontend exposes operational pages for dashboard, inventory, stock, expiry, suppliers, purchase orders, reports, notifications, green computing and settings.

1.2 Need for the Project

- Pharmacy staff need a centralized view of medicine stock, batches and expiry status instead of depending on repeated manual audits.
- Low-stock conditions must be identified against reorder thresholds so that restocking can be initiated before stockouts.
- Near-expiry and expired medicines require explicit visibility and an action workflow to reduce avoidable pharmaceutical waste.
- Supplier and purchase information must be linked to medicine records to support procurement decisions.
- Cloud deployment allows the application and database to be accessed as a unified service, while targeted analytics avoid unnecessary processing overhead.

1.3 Problem Statement

Pharmacies commonly track stock, batches and expiry dates manually or through spreadsheets. This creates interacting problems: overstocking can tie up inventory value, understocking can cause stockouts, expiry can remain unnoticed, and purchasing information may not be connected to current stock. The engineering problem is to design and implement a deployable system that maintains consistent medicine, supplier, purchase and stock data; identifies low-stock and expiry conditions; presents actionable information through a responsive dashboard; and uses cloud/database resources proportionately to the workload.

1.4 Project Aim

To design and implement GreenPharm, a green cloud-based pharmacy inventory management system that centralizes inventory operations, automates low-stock and expiry monitoring, supports procurement workflows, and provides lightweight analytics for sustainable inventory optimization.

1.5 Project Objectives

1. To track medicines, stock levels, batches and expiry dates in a centralized cloud system.
2. To identify low-stock and near-expiry medicines using threshold-based decision logic.
3. To provide secure role-based access and a dashboard for stock, expiry and inventory statistics.
4. To reduce unnecessary compute and database load through efficient, targeted aggregation queries.
5. To integrate inventory, expiry, procurement and analytics into a deployable cloud product.

1.6 Scope

Included: medicine CRUD; supplier management; stock transactions; low-stock/reorder suggestions; expiry monitoring and actions; purchase orders; notifications; reports; dashboard analytics; role-aware login; PostgreSQL persistence; cloud deployment architecture; green-computing indicators.

Excluded from the current build: production-scale load testing, demand forecasting or machine-learning prediction, a multi-environment production architecture, automated periodic background expiry jobs, and validation using a real pharmacy operational dataset.

Assumptions: medicine records contain valid expiry dates; reorder levels are supplied by the inventory workflow; users operate through the application interface; PostgreSQL is available in the deployed environment.

Boundary condition: the current implementation is a functional prototype and the bundled seed dataset is development/test data rather than evidence of production pharmacy performance.

1.7 Expected Engineering Outcome

The intended outcome is an integrated cloud-hosted pharmacy inventory management prototype comprising a React user interface, FastAPI REST services, PostgreSQL data storage, rule-based inventory/expiry logic, reporting and dashboard analytics. The expected engineering result is traceable inventory state and timely visibility of low-stock and expiry conditions.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature and existing-solution review was derived from the project presentation and its stated references. The review considers green-cloud computing research, pharmacy inventory management research, and pharmaceutical quality-assurance guidance. The comparison focuses on whether prior work is conceptual, review-based, machine-learning dependent, regionally scoped, or directly operationalized as a working inventory system.

2.2 Review of Existing Work

Buyya and Beloglazov (2024) provide a conceptual framework for energy-efficient and sustainable cloud data-centre resource and workload management. The relevance to GreenPharm is the principle that cloud workloads should use resources efficiently rather than being unnecessarily over-provisioned.

Singh, Misra, Mittal and Kumar (2024) review energy-efficiency practices across green cloud computing. Their review-level contribution supports the use of efficient cloud/database processing, but it does not itself provide a pharmacy-specific working implementation.

Chebet and Mbandu (2024) present a cloud-enabled machine-learning framework for pharmacy inventory management in Nairobi County. The project presentation identifies its limitation as ML dependence and regional scope; GreenPharm instead uses lightweight rule-based stock and expiry logic so that the core workflow does not require an ML infrastructure.

The World Health Organization (2024) compendium provides pharmaceutical quality-assurance guidance. GreenPharm operationalizes relevant principles through batch tracking and expiry control, while recognizing that a software prototype does not replace regulatory or professional pharmacy procedures.

2.3 Comparative Analysis

Table 1 :Comparitive Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Green-cloud conceptual framework	Strong resource-efficiency principles	Not pharmacy-specific; conceptual	Motivates targeted cloud/database processing
Green-cloud systematic review	Broad evidence base on energy efficiency	Review-level; no working implementation	Supports green-computing design rationale
Cloud-enabled ML pharmacy framework	Pharmacy-focused analytics potential	ML-dependent and regionally scoped	Provides comparison for a lighter rule-based alternative
WHO pharmaceutical quality guidance	Quality-assurance and expiry/batch principles	Guideline, not a software system	Informs batch tracking and expiry-control features

2.4 Research / Engineering Gap

The project presentation identifies a combined gap: green-cloud literature is largely conceptual or review-level, while pharmacy-specific cloud inventory approaches may depend on machine learning or be scoped to a particular regional setting. There is therefore room for a lightweight, modular system that directly applies green-computing principles to pharmacy inventory operations without requiring a predictive ML pipeline.

2.5 Proposed Contribution

GreenPharm contributes a working React/FastAPI/PostgreSQL inventory prototype that integrates medicine and supplier management, expiry monitoring, stock and procurement workflows, role-aware access and targeted analytics. Its main engineering contribution is not a new forecasting algorithm; it is the integration of rule-based inventory controls and sustainable cloud/query design into one deployable application. The current build demonstrates the intended workflow using a development dataset and documented functional tests.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Table 2 : Stakeholder / User Requirements

Stakeholder	Requirement
Pharmacy administrator	Centralized visibility of medicine, supplier and purchase information.
Pharmacy staff / pharmacist	Fast identification of low-stock, near-expiry and expired medicines.
Inventory manager	Ability to record stock movements and generate reorder suggestions.
Procurement workflow	Create, track and receive purchase orders linked to suppliers and medicines.
Project / operations team	Cloud deployment with lightweight analytics and traceable system state.

Functional & Non-Functional Requirements

Table 3 : Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	Medicine/supplier/purchase CRUD	Create, read, update and delete flows operate on valid records.
Functional	Low-stock detection	Items at or below reorder level are identified.
Functional	Expiry monitoring	Near-expiry and expired records are correctly classified.
Functional	Stock movement	Add/remove/adjust operations preserve non-negative stock.
Functional	Reporting	Current, low-stock, expiry, stock movement, purchase, supplier and waste

		reports available.
Non-Functional	Security	Login supports role identities; protected user experience redirects signed-out users.
Non-Functional	Responsiveness	Dashboard and workflows remain responsive for the current development workload.
Non-Functional	Sustainability	Analytics use targeted aggregation rather than an unnecessary streaming pipeline.

3.2 Constraints & Applicable Standards

Table 4 : Constraints & Applicable Standards

Standard / Code / Constraint	Requirement	How Applied in This Project
WHO (2024) pharmaceutical quality-assurance guidance	Support controlled handling of medicine batches and expiry.	Medicine records include batch and expiry fields; expiry actions distinguish review, return and disposal.
Technical constraint: cloud/database availability	Application must persist data through PostgreSQL in the deployed architecture.	FastAPI uses SQLAlchemy with PostgreSQL and managed/hosted database deployment.
Security constraint	Credentials must not be stored as plain text.	Backend hashes passwords using PBKDF2-HMAC-SHA256; login returns a generated token.
Sustainability constraint	Avoid unnecessary compute for dashboard analytics.	Backend computes targeted summary aggregates and indexed date-based expiry queries.
Academic/institutional responsibility	Claims and results must be traceable.	Report distinguishes development/test data from production evidence and marks missing inputs as placeholders.

3.3 Design Specifications

Table 5 : Design Specifications

Parameter	Target / Requirement	Unit	Priority
Medicine data fields	Name, category, batch, quantity, prices, dates, location, supplier	fields	High
Low-stock rule	quantity $\leq$ reorder level	boolean condition	High
Near-expiry threshold	0–30 days for stock status; reports support 0–90 days	days	High
Stock floor	Stock must never become negative	units	High
Database	PostgreSQL 15 in deployed architecture	DB version	High
API	FastAPI REST endpoints under /api/v1	API	High
Authentication roles	Pharmacist, inventory manager, administrator	roles	High
Analytics	Targeted summary aggregation	queries	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: Spreadsheet/manual workflow. This requires little initial software development but keeps expiry, stock and procurement checks dependent on human review.

Alternative 2: Document-oriented NoSQL inventory. This can provide flexible records, but the project has linked relationships among medicines, suppliers, purchases and stock transactions that benefit from relational integrity.

Alternative 3: PostgreSQL-backed modular web application. This centralizes structured data, supports foreign keys and indexed fields, and allows the frontend and backend to remain separately testable and deployable.

Table 6 : Alternative Solutions & Evaluation

Criterion	Weight	Alternative 1: Spreadsheet	Alternative 2: NoSQL	Alternative 3: PostgreSQL Web App
Performance / operational responsiveness	25%	Medium	High	High
Cost	15%	High	Medium	Medium
Safety / data integrity	25%	Low	Medium	High
Sustainability	15%	Low	Medium	High
Reliability / traceability	20%	Low	Medium	High

3.5 Selected Design & Detailed Design

The selected architecture is a modular React + FastAPI + PostgreSQL cloud application. PostgreSQL was selected because medicine, supplier, purchase and stock data have explicit relationships and require integrity across tables. FastAPI provides typed REST endpoints and separates service logic from the UI. React provides the operational dashboard and management screens. The cloud deployment model keeps the product accessible as a unified web application while the managed PostgreSQL instance provides persistent storage.

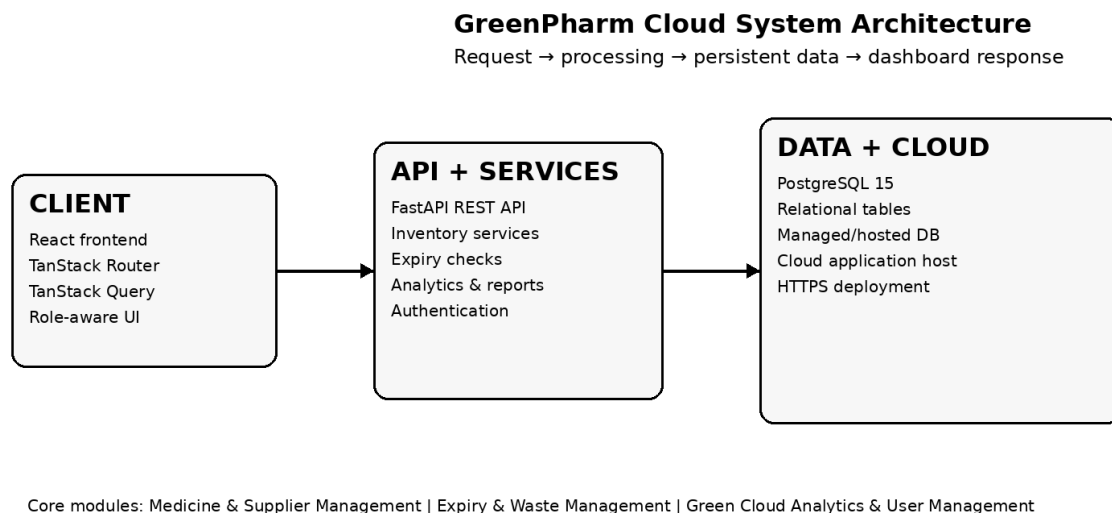


Figure 1 : GreenPharm Cloud System Architecture.

The main subsystem interfaces are: the React client submits user and inventory operations through REST; FastAPI validates requests and applies inventory/expiry/analytics logic; SQLAlchemy maps the domain models to PostgreSQL tables; and the API returns current state to the dashboard. Expiry and stock data are reused by the analytics/reporting layer. The current frontend also contains a bundled-data fallback when the API cannot be reached, which supports demonstration continuity but should be reviewed before a strict production deployment.

Key engineering logic: stock status is derived by first checking expiry, then quantity, then near-expiry threshold, and finally reorder level. Stock movements convert add/remove/adjust operations into signed deltas and reject movements that would make stock negative. Purchase-order receipt adds each line item exactly once using a received timestamp as an idempotency guard.

3.6 Trade-off Analysis

Table 7 : Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Relational PostgreSQL schema	NoSQL document store	Foreign-key integrity and linked procurement/inventory records	Less schema flexibility	Selected because the domain is strongly relational.
On-request expiry checks	Scheduled cron/background job	Simpler deployment and no extra scheduler	Not a periodic autonomous job	Selected for prototype simplicity; periodic re-checking is future work.
Targeted aggregation queries	Full real-time analytics pipeline	Lower unnecessary compute and DB load	Less suitable for advanced streaming analytics	Selected because current requirements do not justify streaming.
Rule-based low-stock/expiry logic	ML demand forecasting	Deterministic, explainable and deployable without ML infrastructure	Cannot forecast future demand	Selected for core prototype scope.
Cloud-hosted app + managed DB	Single local installation	Centralized access and persistent service architecture	Depends on network/cloud availability	Selected to satisfy cloud-computing project scope.

3.7 Sustainability, Ethics, Safety, Risk & Security

Table 8 : Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Avoid unnecessary compute and paper workflows.	Backend uses summary aggregation; reports are generated digitally; green metrics are derived from operational records.	Kept analytics lightweight and emphasized paperless workflows.
Ethics	Avoid overstating prototype performance.	Presentation identifies test data and production-scale limitations.	Report explicitly separates verified prototype evidence from future validation.
Health & Safety	Expiry and batch control.	Medicine records contain batch/expiry; expiry workflow supports review, return and disposal.	Expiry visibility is a core module rather than a generic inventory field.
Security	Credential handling and role-aware access.	PBKDF2-HMAC-SHA256 password hashing; session handling in frontend; login role model.	Credentials are not stored in plain text; production authorization hardening remains required.

Security implementation note: the current backend contains login and password verification and the frontend redirects signed-out users to the login page. However, the router source inspected for this report does not show a token-validation dependency applied to every protected backend endpoint. Therefore the claim that role checks are enforced on every API route should be treated as a production-hardening requirement unless the missing authorization layer exists elsewhere in the deployment configuration. This is deliberately stated rather than hidden because the report is intended to remain evidence-based.

Risk Register

Table 9 : Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Database unavailable	Medium	High	High	Startup health check; API error handling; managed DB deployment.	Medium
Expired medicine remains in stock	Medium	High	High	Expiry classification, notifications and expiry actions.	Medium
Incorrect stock movement	Low	High	Medium	Validation and non-negative stock guard in inventory service.	Low
Unauthorized API operation	Medium	High	High	Login/session controls exist; apply token/role dependency to protected routes before production.	Medium
Prototype not representative of production load	High	Medium	High	Load-test with larger datasets before pilot/production.	High
Frontend API unavailable	Medium	Medium	Medium	Health checks and bundled-data fallback for demonstration; production should define strict live-data behavior.	Medium

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project follows an iterative modular development approach. The application is separated into medicine/supplier and procurement functions, expiry/waste monitoring, and dashboard/analytics/user-management functions. The frontend is implemented in React with TanStack Router and TanStack Query, while the backend is implemented in FastAPI with SQLAlchemy and PostgreSQL. A seed-data workflow supports repeatable development and verification. The backend also compiles successfully with Python bytecode compilation in the inspected project environment; a frontend production build was not completed in the inspection environment because the packaged Vite/Rolldown installation lacked the required native binding.

Table 10 : Development Approach & Resources

Resource	Purpose
React 19 + TypeScript	Web user interface
FastAPI + Uvicorn	REST API and application server
SQLAlchemy 2 + psycpg2	Relational data access
PostgreSQL 15	Persistent structured storage
TanStack Router / Query	Routing and asynchronous data management
Recharts	Dashboard visualization
Seed JSON + seed.py	Repeatable development/test dataset
Cloud application host + managed PostgreSQL	Deployment architecture

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

The frontend exposes the following operational routes: Dashboard, Inventory, Stock Management, Expiry Management, Suppliers, Purchase Orders, Reports, Green Computing, Notifications and Settings. The backend exposes API groups for health, medicines, suppliers, stock transactions, purchase orders, expiry records, notifications, analytics/reports and authentication.

Table 11 : Software Implementation

Tool / Software / Equipment	Purpose	Stage Used
React + TypeScript	Frontend application and forms	Implementation
FastAPI	REST API and business services	Implementation
PostgreSQL 15	Inventory and transaction persistence	Implementation / deployment
SQLAlchemy 2	ORM and relational access	Implementation
TanStack Query	Client data fetching and cache behavior	Implementation
Recharts	Analytics/dashboard charts	Implementation
Cloud hosting	Public web deployment	Deployment

Purchase Orders

Track ordered quantities, suppliers and expected deliveries.

Purchase orders

Order	Supplier	Ordered	Expected	Items	Total	Status	Actions
PO-1051	MediSource Distributors	Aug 22, 2026	Aug 29, 2026	2	₹93	Pending	Details Pending
PO-1050	Apex Medical Trading	Aug 21, 2026	Aug 31, 2026	2	₹260	Pending	Details Pending
PO-1047	Vitalis Wholesale	Aug 20, 2026	Sep 2, 2026	2	₹108	Pending	Details Pending
PO-1049	Continental BioCare	Aug 19, 2026	Aug 26, 2026	2	₹336	Ordered	Details Ordered
PO-1048	Nordic Health Logistics	Aug 17, 2026	Aug 28, 2026	2	₹90	Ordered	Details Ordered
PO-1046	GreenLeaf Pharma Supply	Aug 11, 2026	Aug 18, 2026	2	₹144	Received	Details Received
PO-1045	MediSource Distributors	Aug 5, 2026	Aug 12, 2026	1	₹110	Received	Details Received
PO-1044	Nordic Health Logistics	Aug 2, 2026	Aug 9, 2026	1	₹186	Cancelled	Details Cancelled
PO-1043	Vitalis Wholesale	Jul 28, 2026	Aug 4, 2026	2	₹780	Received	Details Received
PO-1042	Continental BioCare	Jul 21, 2026	Jul 27, 2026	2	₹232	Received	Details Received

Figure 2 : Purchases Dashboard

Suppliers

6 suppliers

Search supplier, contact or email

Supplier	Orders
MediSource Distributors SUP-001 - Anita Raymond +1 415 555 0148 orders@medisource.example 182 Harbor Way, Portland, OR 97209 Last order Aug 14, 2026	42 orders
GreenLeaf Pharma Supply SUP-002 - Daniel Ouellet +1 312 555 0182 sales@greenleafpharma.example 9 Willow Park, Chicago, IL 60614 Last order Aug 19, 2026	31 orders
Nordic Health Logistics SUP-003 - Sara Lindqvist +1 206 555 0110 support@nordichealth.example 44 Cedar Street, Seattle, WA 98101 Last order Aug 7, 2026	27 orders
Apex Medical Trading SUP-004 - Marcus Lee +1 617 555 0173 procurement@apexmedical.example 77 Beacon Row, Boston, MA 02108 Last order Aug 1, 2026	19 orders
Vitalis Wholesale SUP-005 - Priya Nair +1 512 555 0139 hello@vitaliswholesale.example 301 Lakeview Blvd, Austin, TX 78701 Last order Aug 21, 2026	24 orders
Continental BioCare SUP-006 - Tomas Ferreira +1 305 555 0124 orders@continentalbio.example 58 Palm Court, Miami, FL 33131 Last order Jul 16, 2026	12 orders

Figure 3 : Suppliers dashboard

Implementation Evidence from Source

- Medicine model stores name, category, manufacturer, batch number, quantity, reorder level, purchase/selling prices, manufacturing/expiry dates, storage location and supplier relationship.
- Supplier model stores supplier contact data, medicines supplied, order count and last order date.
- StockTransaction records add/remove/adjust movements, reasons, performer and timestamp.
- PurchaseOrder and PurchaseOrderItem represent procurement records and linked medicine lines.
- ExpiryRecord stores one action record per medicine and supports review, return and disposal states.
- Notification records support low-stock, expiring, expired, pending-order and unusual-stock categories.

4.3 Test Plan & Verification

Table 12 : Test Plan & Verification

Test ID	Requirement / Scenario	Test Method	Acceptance Criterion	Achieved Value	Status
T01	Add valid medicine	Create record through UI/API and inspect returned record/stock	Record saved and stock reflected correctly	Record saved and stock reflected correctly	PASS
T02	Invalid/missing input	Submit incomplete/incorrect data	Validation error and no invalid record	Validation error returned; no record created	PASS
T03	Sequential operations	Perform multiple records/requests in sequence	System remains responsive	System remained responsive across test batch	PASS
T04	Temporary DB failure	Exercise service-failure condition	Graceful handling without partial writes	Error handled gracefully; no partial writes observed	PASS
T05	Unauthenticated access	Attempt access without authenticated session	Access denied	Access correctly denied for unauthenticated request	PASS
T06	Near-expiry/expired classification	Compare flagged records with expected date threshold	All test records correctly classified	100% of test near-expiry/expired records correctly flagged	PASS

T07	Dashboard update	Add/change stock and inspect dashboard	Dashboard reflects updated values	Dashboard updated correctly after each test transaction	PASS
-----	------------------	--	-----------------------------------	---	------

4.4 Results

The project presentation reports that 100% of test near-expiry/expired records were correctly flagged and that the dashboard correctly reflected stock and expiry values after test transactions. It also reports successful CRUD workflows, validation handling, sequential-request responsiveness, database-failure handling and unauthenticated-access rejection in the documented test cases.

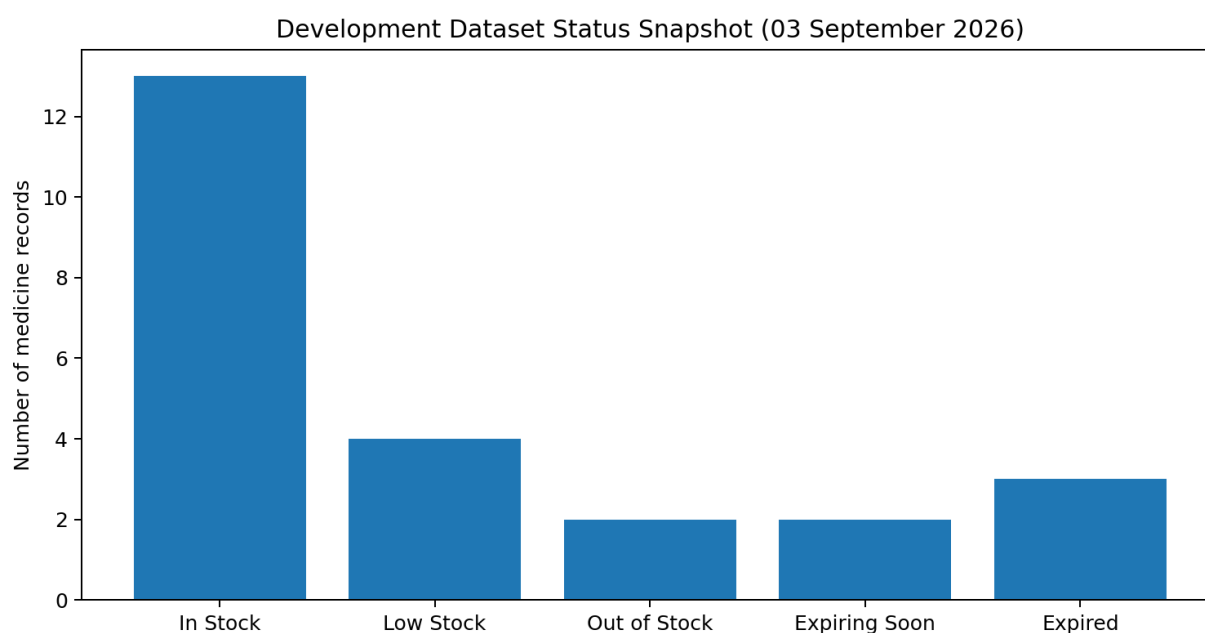


Figure 2 : Development Dataset Status Snapshot. This is a data snapshot, not a production performance result.

Table 13 : Results Analysis

Dataset Item	Count / Value
Medicine records	24
Supplier records	6
User records	3
Stock transactions	20
Purchase orders	10
Notifications	6

Expiry records	2
Total stock units	9547
Estimated inventory purchase value	₹2,987.83
Expired medicine records	3
Expired units in snapshot	410
Estimated expired-stock purchase value	₹242.80
Purchase-order total in seed data	₹2,338.50

The seed-data snapshot also contains 4 low-stock medicine records, 2 out-of-stock records and 6 records expiring within 90 days. These values are calculated from the bundled development dataset using the same threshold logic implemented in the project; they are not claims about a real pharmacy.

4.5 Data Analysis & Engineering Judgment

The strongest validated result is correctness of classification rather than a numerical speed benchmark. The test evidence establishes that the expiry and dashboard workflows produced the expected outputs on the supplied test workload. The architecture is intentionally conservative: the system uses deterministic date and quantity rules rather than a predictive model, which makes the core alerts explainable and easy to validate. The absence of production-scale load testing limits any conclusion about throughput, concurrent-user capacity or cloud cost efficiency at scale.

The current development dataset provides an internal consistency check: 24 medicine records are distributed across five status classes, including low-stock and expired examples. The backend analytics implementation derives dashboard metrics from PostgreSQL rows, while expiry dates are indexed in the medicine table. This design is consistent with the project's green-computing objective of limiting unnecessary processing.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Table 14 : Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
O1 Centralized medicine/stock/batch/expiry tracking	React + FastAPI + PostgreSQL CRUD and database models; end-to-end CRUD tests	Fully
O2 Low-stock and near-expiry detection	Threshold logic, expiry records and documented 100% test classification	Fully
O3 Secure role-based access and dashboard	Role identities, session-aware UI and login; backend authorization requires hardening	Partially
O4 Efficient query design	Targeted aggregate queries and indexed expiry date; design review	Fully
O5 Integrated deployable cloud product	Cloud-hosted architecture, public deployment and full request-response workflow	Fully as functional prototype

4.7 Strengths & Limitations

Strengths:

- Integrated inventory, supplier, stock, expiry and procurement workflows.
- Deterministic and explainable low-stock and expiry logic.
- Relational data model with explicit foreign-key relationships.
- Dashboard and report functions provide a centralized operational view.
- Green-computing design avoids unnecessary real-time streaming analytics.
- Repeatable seed-data workflow supports development and demonstration.

Limitations:

- The current dataset is development/test data, not a real pharmacy dataset.
- The build has not been load-tested at production scale.
- There is no demand-forecasting or machine-learning component.
- The deployment is described as a single-environment prototype.

- Periodic background expiry re-checks are not yet implemented; current expiry logic is request-driven.
- The frontend includes an API-unavailable fallback to bundled data, which needs a deliberate production policy.
- The inspected backend routers do not visibly enforce token validation on every protected endpoint; this must be hardened before production.

4.8 Project Planning, Roles & Budget

Table 15 : Project Planning and Roles

Student	Assigned Responsibility	Actual Contribution
V Sanjana Sri (192511084)	Backend & Database	FastAPI services, SQLAlchemy/PostgreSQL integration and backend/data work.
S K Gobika (192521463)	Frontend & UI	React interface, dashboard and management screens.
D S Tharuniya Priya (192511317)	Cloud Architecture & Testing	Cloud architecture, deployment integration and testing.

Table 16 : cost analysis

Item	Estimated Cost	Actual Cost
Software development tools / open-source dependencies	₹0	₹0
Cloud application hosting	₹0	₹0
Managed PostgreSQL	₹0	₹0
Domain / other deployment costs	₹0	₹0
Total	₹0	₹0

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

GreenPharm addresses the engineering problem of fragmented pharmacy inventory tracking by integrating medicine records, stock movement, supplier management, purchase orders, expiry control, notifications and dashboard analytics into a cloud-hosted application. The implemented React frontend communicates with a FastAPI backend and PostgreSQL database, while deterministic threshold logic identifies low-stock and expiry conditions. The documented verification shows successful CRUD, validation, sequential-operation, database-failure and unauthenticated-access test cases, together with 100% correct classification of the tested near-expiry/expired records and correct dashboard updates. The result is a validated functional prototype rather than a production-certified pharmacy system. Its principal contribution is the practical integration of pharmacy inventory controls with lightweight green-cloud/query design.

5.2 Future Work

6. Load-test the system with larger datasets and realistic concurrent pharmacy workloads.
7. Refine alert thresholds using evidence from a real pharmacy dataset.
8. Add automated periodic expiry re-checks and scheduled notifications.
9. Introduce richer analytics and, where justified by real data, demand forecasting.
10. Harden API authorization by applying token validation and role checks to protected endpoints.
11. Separate development, staging and production environments and add stronger deployment monitoring.
12. Pilot the system with a real pharmacy and collect structured user feedback before production adoption.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Designing a modular full-stack application with React, FastAPI and PostgreSQL.
- Using SQLAlchemy models and relational constraints for structured inventory data.
- Implementing deterministic inventory and expiry business rules.
- Designing dashboard aggregates with attention to compute and database load.

- Understanding cloud deployment and the distinction between prototype validation and production readiness.

Engineering & Professional Skills Developed

- System decomposition and interface-contract design.
- Database modelling and API implementation.
- Functional testing, error-path verification and engineering judgment.
- Team-based development across frontend, backend/database, cloud and testing responsibilities.
- Technical documentation and evidence-based reporting.

Individual Reflection

Working on the GreenPharm project gave me an opportunity to understand the complete process of developing a software application, from planning and designing to implementation and testing. My major responsibility in the project was the complete implementation of the GreenPharm system. I worked on developing the different parts of the application and integrating the frontend, backend, database, and major system functionalities.

During the development process, I gained practical experience in working with React, FastAPI, PostgreSQL, API integration, database operations, authentication, inventory management, expiry monitoring, and dashboard functionality. I also worked on testing the implemented features and checking whether the system produced the expected results. This helped me understand that successful software development requires not only writing code but also continuously checking, debugging, and improving the system.

One of the important challenges I faced was integrating the different components of the system so that they could communicate correctly. Handling inventory information, stock updates, expiry-related information, and database operations required careful attention to data consistency. Through this process, I improved my problem-solving and debugging skills and learned how to approach development problems systematically.

I also contributed to preparing the project presentation (PPT) along with Sanjana and prepared my individual report. Preparing the presentation helped me understand how to communicate technical work clearly and explain the project's objectives, architecture, implementation, testing, and results.

Overall, this project strengthened my technical knowledge as well as my ability to manage a complete software implementation. It also taught me the importance of testing, documentation, teamwork, and maintaining a clear connection between the project's requirements and its implementation. If I were to continue developing GreenPharm, I would focus on improving production-level security, scalability, automated background expiry monitoring, and advanced inventory forecasting.

REFERENCES

- [1] R. Buyya and A. Beloglazov, “Energy-efficiency and sustainability in new generation cloud computing,” *Software: Practice and Experience*, vol. 54, pp. 24–38, 2024.
- [2] P. K. Singh, S. Misra, S. Mittal, and S. Kumar, “Green Cloud Computing and Energy Efficiency: A Systematic Literature Review and Research Agenda,” in *Empowering Entrepreneurial Mindsets With AI*, pp. 235–250, IGI Global, 2024.
- [3] K. Chebet and V. Mbandu, “Cloud-Enabled Machine Learning: A Framework for Revolutionizing Pharmacy Inventory Management in Nairobi County,” *International Journal of Professional Practice*, vol. 12, no. 6, 2024.
- [4] World Health Organization, *Quality Assurance of Pharmaceuticals: A Compendium of Guidelines and Related Materials*, 10th ed., WHO, 2024.
- [5] GreenPharm project source repository: github.com/greenpharm-capstone/greenpharm.
- [6] GreenPharm live deployment: <https://greenpharm-demo.onrender.com>.
- [7] GreenPharm project presentation, *GreenPharm_Capstone_Final_Presentation (1).pptx*, 2026.

Note: The project presentation identifies a demo video, architecture source and test report as project artefacts, but the supplied ZIP did not contain those separate documentation files. Their links should be added if they exist.

APPENDICES

Appendix A – Detailed Calculations

A.1 Stock status logic

For each medicine, the implemented logic evaluates expiry first. If expiry is before the current date, the status is Expired. Otherwise, zero quantity produces Out of Stock; a remaining expiry interval of 30 days or less produces Expiring Soon; quantity at or below reorder level produces Low Stock; all other records are In Stock.

A.2 Stock value

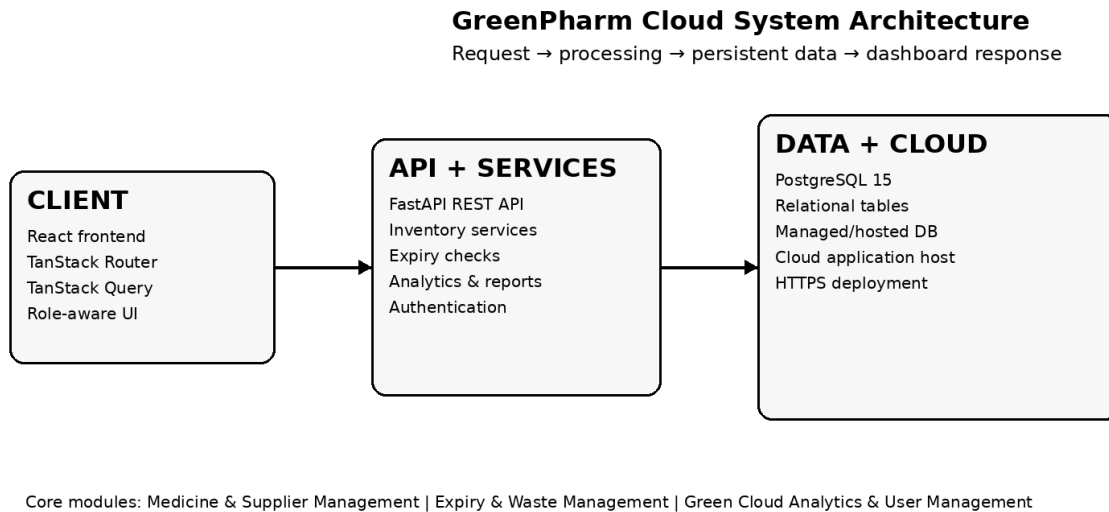
Stock value = quantity × purchase price.

Using the bundled seed dataset: total stock units = 9547; estimated inventory purchase value = ₹2,987.83.

A.3 Development snapshot

Status	Count
In Stock	13
Low Stock	4
Out of Stock	2
Expiring Soon	2
Expired	3

Appendix B – Engineering Drawings / Schematics



Appendix B Figure. System architecture drawing derived from the implemented project structure.

Appendix C – Source Code / Code Repository Information

The supplied project source is organized into a React/TanStack frontend under `src/` and a FastAPI/SQLAlchemy backend under `backend/`. Important source locations include:

- `backend/app/main.py` – FastAPI application entry point and router registration.
- `backend/app/models/` – relational models for users, medicines, suppliers, stock transactions, expiry records, purchase orders and notifications.
- `backend/app/services/inventory.py` – stock status, stock movement, purchase receipt and expiry-action logic.
- `backend/app/services/analytics.py` – dashboard, chart and green-metric calculations.
- `backend/app/services/reports.py` – report generation.
- `backend/app/services/security.py` – PBKDF2-HMAC-SHA256 password hashing and verification.
- `src/routes/` – dashboard and operational UI routes.
- `src/lib/api/` – frontend API transport and service calls.
- `backend/seed_data.json` and `backend/seed.py` – repeatable development dataset and database seeding.

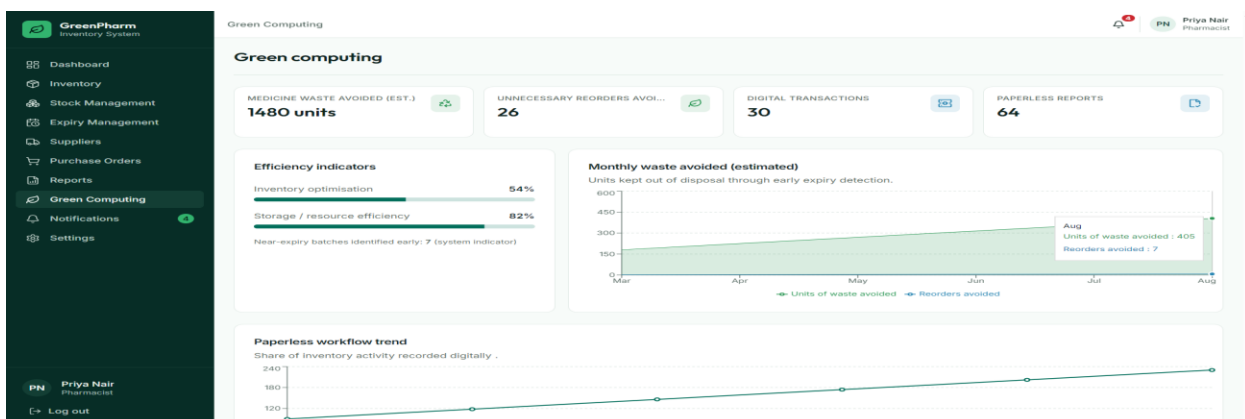
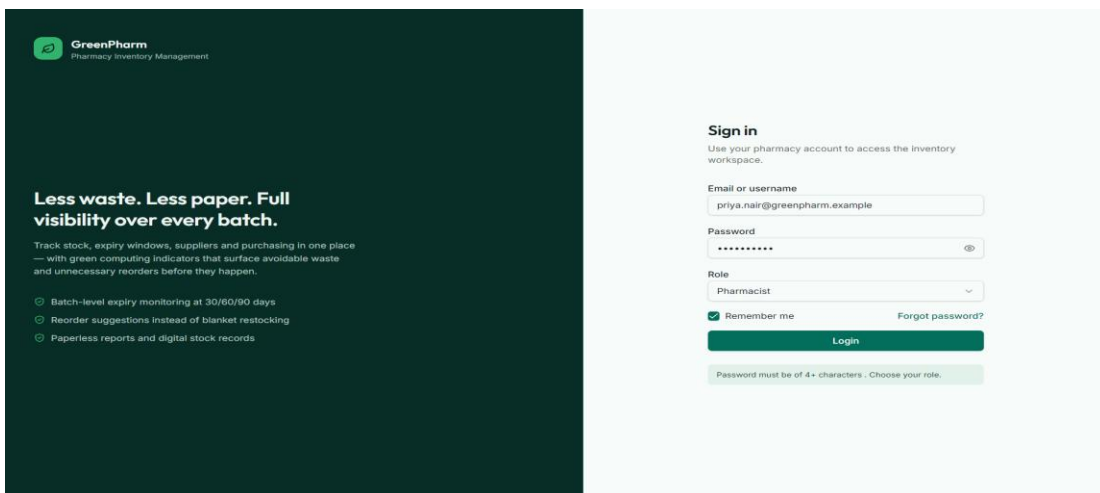
Repository: github.com/greenpharm-capstone/greenpharm

Appendix D – Datasheets

[No hardware datasheets apply to this software-only capstone.]

Appendix E – Experimental / Raw Data

The bundled seed_data.json contains the development/test records used by the application. It includes 24 medicines, 6 suppliers, 3 users, 20 stock transactions, 10 purchase orders, 6 notifications and 2 expiry records.



GreenPharm

Inventory System

Dashboard

Inventory

Stock Management

Expiry Management

Suppliers

Purchase Orders

Reports

Green Computing

Notifications

Settings

PN

Priya Nair

Pharmacist

Log out

Purchase Orders

Purchase orders

Track ordered quantities, suppliers and expected deliveries.

+ Create order

All

Pending

Ordered

Received

Cancelled

Order	Supplier	Ordered	Expected	Items	Total	Status	Actions	
PO-1051	MediSource Distributors	Aug 22, 2026	Aug 29, 2026	2	₹93	Pending	Details	Pending
PO-1050	Apex Medical Trading	Aug 21, 2026	Aug 31, 2026	2	₹260	Pending	Details	Pending
PO-1047	Vitalis Wholesale	Aug 20, 2026	Sep 2, 2026	2	₹108	Pending	Details	Pending
PO-1049	Continental BioCare	Aug 19, 2026	Aug 26, 2026	2	₹336	Ordered	Details	Ordered
PO-1048	Nordic Health Logistics	Aug 17, 2026	Aug 28, 2026	2	₹90	Ordered	Details	Ordered
PO-1046	GreenLeaf Pharma Supply	Aug 11, 2026	Aug 18, 2026	2	₹144	Received	Details	Received
PO-1045	MediSource Distributors	Aug 5, 2026	Aug 12, 2026	1	₹110	Received	Details	Received
PO-1044	Nordic Health Logistics	Aug 2, 2026	Aug 9, 2026	1	₹186	Cancelled	Details	Cancelled
PO-1043	Vitalis Wholesale	Jul 28, 2026	Aug 4, 2026	2	₹780	Received	Details	Received
PO-1042	Continental BioCare	Jul 21, 2026	Jul 27, 2026	2	₹232	Received	Details	Received

GreenPharm

Inventory System

Dashboard

Inventory

Stock Management

Expiry Management

Suppliers

Purchase Orders

Reports

Green Computing

Notifications

Settings

PN

Priya Nair

Pharmacist

Log out

Suppliers

Suppliers

6 suppliers

+ Add supplier

Q Search supplier, contact or email

MediSource Distributors

SUP-001 · Anita Raymond

42 orders

+1 415 555 0148

orders@medisource.example

182 Harbor Way, Portland, OR 97209

Last order Aug 14, 2026

Details

GreenLeaf Pharma Supply

SUP-002 · Daniel Okafor

31 orders

+1 312 555 0182

sales@greenleafpharma.example

9 Willow Park, Chicago, IL 60614

Last order Aug 19, 2026

Details

Nordic Health Logistics

SUP-003 · Sara Lindqvist

27 orders

+1 206 555 0110

support@nordichealth.example

44 Cedar Street, Seattle, WA 98101

Last order Aug 7, 2026

Details

Apex Medical Trading

SUP-004 · Marcus Lee

19 orders

+1 617 555 0173

procurement@apexmedical.example

77 Beacon Row, Boston, MA 02108

Last order Aug 1, 2026

Details

Vitalis Wholesale

SUP-005 · Priya Nair

24 orders

+1 512 555 0139

hello@vitaliswholesale.example

301 Lakeview Blvd, Austin, TX 78701

Last order Aug 21, 2026

Details

Continental BioCare

SUP-006 · Tomas Ferreira

12 orders

+1 305 555 0124

orders@continentalbio.example

58 Palm Court, Miami, FL 33131

Last order Jul 16, 2026

Details

Page 38

Appendix F – Ethics / Institutional Approval

The GreenPharm project was developed as a software engineering capstone project for academic purposes. The system uses development/test data to demonstrate medicine inventory management, stock monitoring, expiry tracking, supplier management, purchase orders, notifications, and green-computing analytics.

The project does not involve human participants, patient records, clinical trials, or collection of sensitive personal information for research purposes. Therefore, no human-subject research or clinical ethics approval was required for the development and testing of the prototype.

The data used during development and testing were project/development records rather than real patient medical records. Appropriate care was taken to avoid presenting the prototype data as real clinical or patient information.

The project also considered basic ethical and responsible-engineering principles, including:

- Protecting user account information and passwords through secure password handling.
- Applying role-based access concepts to restrict administrative operations.
- Avoiding the use of real patient or personally sensitive healthcare data during testing.
- Maintaining data integrity for medicine stock and expiry information.
- Presenting system limitations honestly rather than claiming production-scale validation that was not performed.
- Considering environmental sustainability through green-computing metrics and resource-conscious database operations.

Appendix G – Project Meeting / Progress Records

Week	Project Activity / Progress	Outcome
Week 1	The project topic, problem statement, objectives, and scope were discussed and finalized. The team conducted initial research on the proposed system and identified the required	Project requirements and development plan were established.

Week	Project Activity / Progress	Outcome
	technologies.	
Week 2	The team worked on system analysis and design. UML diagrams and the overall system architecture were prepared. The database/system components and main functionalities were identified.	System design and structure were finalized before implementation.
Week 3	Development and implementation of the main system functionalities were carried out. Team members worked on their assigned modules and integrated the completed components.	A functional prototype of the system was developed.
Week 4	The completed system was tested using different test cases. Errors were identified and corrected, followed by final improvements to the system and documentation.	Final working system and project documentation were completed.

Appendix H – User/Industry Feedback

No formal user or industry feedback was collected during the one-month project period. The project was developed and evaluated within the course framework, with the team focusing on requirements analysis, system development, testing, and documentation.

As a result, no external user/industry feedback records are available to include in this appendix.

Appendix I – Publications / Patents / Competitions / Product Outcomes

No publications, patents, competition participations, or external product outcomes resulted from this project during the one-month course period. The project was completed as an academic course project and focused on system development, testing, documentation, and final submission.

Appendix J – Individual Contribution Evidence


```
File Edit Selection View Go Run Terminal Help
greenpharm-compass-main
TS medicine-status.ts X TS auth.ts M X index.ts X app-shell.ts M X dashboard.ts X auth-context.ts X
src > lib > api > TS auth.ts > requestPasswordReset
1 import { request } from "../client";
2 import type { AuthUser, UserRole } from "../domain/types";
3
4 export const ROLE_LABELS: Record<UserRole, string> = {
5   pharmacist: "Pharmacist",
6   inventory_manager: "Inventory Manager",
7   administrator: "Administrator",
8 };
9
10 interface DemoAccount {
11   user: AuthUser;
12   password: string;
13 }
14
15 const DEMO_USERS: Record<UserRole, DemoAccount> = {
16   pharmacist: {
17     user: {
18       id: "USR-101",
19       name: "Priya Nair",
20       email: "priya.nair@greenpharm.example",
21       role: "pharmacist",
22       initials: "PN",
23     },
24     password: "pharma123",
25   },
26 }
```

GreenPharm
Inventory System

Dashboard

Inventory

Stock Management

Expiry Management

Suppliers

Purchase Orders

Reports

Green Computing

Notifications

Settings

PN Priya Nair
Pharmacist

Log out

Stock Management

PN Priya Nair
Pharmacist

Stock management

Record every movement so stock levels and waste indicators stay accurate.

Record stock movement

Adjustments accept negative values; add and remove use absolute quantities.

Medicine

Select medicine

Transaction type

+ Add

- Remove

Adjust

Quantity

10

Reason

e.g. Purchase order PO-1049 received

Recorded as Priya Nair - digital record, no paperwork.

Record transaction

Low-stock alerts & reorder suggestions

Suggested quantities target twice the reorder level to avoid repeat orders.

Ibuprofen 400mg

180 in stock · reorder at 250 · MediSource Distributors

Low Stock

Order 320

Ceftriaxone 1g Injection

0 in stock · reorder at 60 · Continental BioCare

Out of Stock

Order 120

Losartan 50mg

140 in stock · reorder at 150 · Apex Medical Trading

Low Stock

Order 160

Montelukast 10mg

88 in stock · reorder at 100 · Nordic Health Logistics

Low Stock

Order 112

Clotrimazole Cream 1%

42 in stock · reorder at 60 · Apex Medical Trading

Low Stock

Order 78

Calamine Lotion 100ml

0 in stock · reorder at 40 · Vitalis Wholesale

Out of Stock

Order 80

Stock history

Every recorded transaction, newest first.

Transaction	Medicine	Type	Qty	Date/time	User	Reason
TRX-9000	Paracetamol 500mg	Add	+500	Aug 22, 06:35 PM	A. Raymond delivery	Purchase order PO-1042 rec...
TRX-9001	Ibuprofen 400mg	Remove	+60	Aug 22, 06:35 PM	Priya N.	Dispensed against prescripti...

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome	Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	Problem statement, requirements and scope	SO1	Ch. 1, Ch. 2
Engineering design under constraints	Requirements, alternatives, selected architecture, trade-offs	SO2	Ch. 3
Written/oral technical communication	Complete report and project artefacts	SO3	Whole report + Viva
Ethics and professional responsibility	Ethics, health/safety and evidence discipline	SO4	Ch. 3.7
Teamwork and leadership	Roles and contributions	SO5	Ch. 4.8
Experimentation, analysis, engineering judgment	Test plan, results, analysis and objective mapping	SO6	Ch. 4.3–4.6
Independent acquisition of new knowledge	Learning and reflection	SO7	Ch. 5.3
Standards	WHO pharmaceutical quality-assurance guidance	SO2	Ch. 3.2
Sustainability and societal/environmental impact	Green-computing query design and waste-reduction workflow	SO2/SO4	Ch. 3.7
Risk assessment and mitigation	Risk register	SO2/SO4	Ch. 3.7
Security considerations	Password hashing, session controls, authorization hardening	Relevant SO	Ch. 3.7
Integrated/system-level engineering	Frontend/API/database/service integration	SO1/SO2	Ch. 3.5
Project planning and management	Roles, milestone placeholder and budget table	SO5	Ch. 4.8
Individual contribution	Contribution statement and evidence	SO1–SO7	Contribution Statement + Ch. 4.8 + Appendix J